

Music format - Pilotwings 64 (USA)

Findings behind the Music tab of Hawk's Hangar. The game uses the N64 SDK audio library (libaudio), not the EAD driver of Super Mario 64, so there are no .m64 files.

Audio block (behind the file system, addresses hardcoded in the audio code)

Part	ROM offset	Size	Content
Sequences (ALSeqFile "S1")	0x618B70	84,208	31 songs, compact MIDI
Instrument bank (.ctl, "B1")	0x62D460	16,496	1 bank, 100 places, 48 instruments, 127 sounds
Samples (.tbl)	0x6314D0	833,296	VADPCM, 22,050 Hz

S1 file: "S1", u16 count, then (u32 offset, u32 length) per song; songs 4-byte aligned, file padded to 16. The game copies one song at a time into a buffer as large as the longest song.

Compact MIDI (ALCSeq, alCSPlayer)

- Header: $16 \times \text{u32 track offsets (0 = unused) + u32 division (768 in all songs)}$.
- Track 0 of every song only holds tempo; each other track plays exactly one MIDI channel.
- Note-on is followed by the note length (varlen); there are no note-off events.
- $\boxed{\text{FE hi lo n}}$: repeat n earlier bytes starting (hi·256+lo) bytes before the code; may point into other tracks. $\boxed{\text{FE FE}}$ = literal 0xFE. Repeated bytes are read raw.
- Loops: $\boxed{\text{FF 2E xx xx}}$ (start), $\boxed{\text{FF 2D}}$ + 6 raw bytes (count, current count, u32 distance back).

Instruments

- Program numbers index the bank directly (not General MIDI). Empty places crash the game when a note is played, so the editor only offers the 48 used ones.
- Channel 10 (index 9) starts with the bank's percussion instrument = program 1 (Drums 1); all other channels start with the first instrument (program 0). Songs 11 and 20 play drums on channel 10 without a program change.
- Instrument names and categories were given by the project owner after listening (2026-10-09).

Editor behavior

- Instrument change = program numbers changed in place; mute = volume (CC 7) values set to 0.
- A byte shared by repetitions with different needs is written out (the repetition expanded); missing program/volume events are inserted at the track's first event, and all track offsets, repetition distances and loop distances across the change are corrected.
- Every result is re-parsed and compared event by event with the expected result.

- Changes are stored per song in project.json (`Music`); the build writes a new S1 file. If it grows, the bank and samples move behind it and the code references are patched.

Loops in the original songs

- All looping songs loop as a whole: every track has `FF 2E 00 FF` (loop start) and `FF 2D FF FF <u32>` (loop end, count 255 = forever) at the same ticks. The jump goes to the time value right behind the loop start event. Jingles (landing, crash, ...) have no loop.

MIDI export and import

- Export: MIDI type 1, track 1 = tempo + markers `loopStart`/`loopEnd`, one track per channel (named after voice and instrument). Program numbers are the game's instruments. The current project version is exported (imported song + instrument changes + mutes).
- Import: type 0 and 1, ticks per quarter note. Loop from markers `loopStart`/`loopEnd` (also "loop start", "[", "]") or CC 102/103. Notes get their length from the note-off; notes behind the loop end are dropped; unknown programs fall back to the channel's default instrument (with a warning); sysex is dropped. One track per channel, tempo in the first.
- Packing: running status plus repeated runs (`FE hi lo n`), at least 16 bytes, sources only plain bytes, never loop data, no 0xFE). Re-importing all exported songs gives 83,627 bytes in total (originals: 83,902) with identical events.
- Size limit: the longest original song (9,834 bytes), because the game reserves a buffer of that size in its audio heap (0x413DC bytes) at start-up. Also checked when building.
- Imported songs are stored in `files/music/song_NN.seq` (game format), so restore points keep them.